

Nome: Guilherme de Moraes China Costa

Data: 08/12/2025

Título: Engenharia de Software: Clean Code e Boas Práticas - AT

Link implementação:

https://github.com/guilhermecosta/INFNET/tree/master/clean_code/AT

Parte 1 - Análise Crítica do Código Legado

Principais problemas encontrados

O código original da classe Pedido tem vários pontos críticos que dificultam a vida de quem precisa mexer nele. Os principais são:

1. **Campos públicos:** endereço, peso, tipoFrete e destinatario são public. Qualquer um pode alterar direto e deixar o objeto em estado inválido (peso negativo, endereço nulo, etc.).
2. **Mistura de responsabilidades:** A mesma classe guarda os dados, calcula frete, formata etiquetas e resumos, aplica promoção e verifica frete grátis. É o famoso “God Object” – faz tudo e não faz nada bem.
3. **Lógica duplicada:** O método calcularFrete() é chamado em vários lugares (etiqueta, resumo). Se precisar mudar a fórmula, tem que lembrar de alterar em todos os pontos.
4. **Valores mágicos por toda parte:** “EXP”, “PAD”, “ECO”, 1.5, 10, 1.2, 1.1, -5... ninguém sabe o que significam sem ler o código inteiro.
5. **Zero validação:** Aceita peso negativo, tipo de frete errado, endereço vazio ou nulo e simplesmente devolve frete zero sem avisar ninguém.
6. **If-else gigante e rígido:** Todo o cálculo de frete está preso num bloco de if/else. Para adicionar um novo tipo de frete é preciso abrir a classe e mexer no meio do código (violação direta do Open-Closed Principle).
7. **Nomes confusos:** aplicarFretePromocional() na verdade diminui o peso (efeito colateral estranho), tipoFrete é uma String mágica, e alguns métodos não seguem o padrão Java.
8. **Sem encapsulamento nem imutabilidade:** O objeto pode ser alterado a qualquer momento por qualquer classe, o que gera bugs difíceis de rastrear.

Resumindo: o código funciona, mas é frágil, difícil de entender, impossível de testar com tranquilidade e doloroso de evoluir.

Parte 2 - O que foi feito na refatoração (passo a passo)

A refatoração seguiu as etapas pedidas, usando Java 17 + Maven. A estrutura ficou organizada em pacotes claros:

- com.infnet.domain → entidades
- com.infnet.frete → estratégias de cálculo e promoções
- com.infnet.service → serviços/orquestração
- com.infnet.exception → exceções do domínio

Passo 1 – Separação de responsabilidades

- A classe Entrega virou imutável (atributos private final, tudo validado no construtor).
- Cálculo de frete saiu da entidade e foi para a interface CalculadoraFrete com três implementações: FreteExpresso, FretePadrao e FreteEconomico.
- Geração de etiqueta e resumo foi para o EtiquetaService, que recebe as estratégias por injeção.

Passo 2 – Validações e tratamento de erro

- Construtor de Entrega agora lança exceções específicas (InvalidPesoException, InvalidEnderecoException, InvalidTipoFreteException) assim que detecta dados ruins.
- Nada mais de retorno silencioso de frete zero.

Passo 3 – Preparação para extensão futura

- Strategy Pattern elimina completamente o if-else gigante.
- Para adicionar um novo tipo de frete basta criar uma nova classe que implemente CalculadoraFrete.
- Promoções também viraram estratégia (PromocaoFrete + PromocaoPesoAlto).
- Seleção da estratégia correta pode ser feita por fábrica (FreteFactory) ou injeção direta.

Testes

Foram criados testes unitários com JUnit cobrindo:

- criação válida e inválida de Entrega
- cálculo correto de cada tipo de frete
- aplicação de promoção
- geração de etiqueta e resumo
- verificação de frete grátis

Agora o código está limpo, fácil de entender, fácil de testar e, principalmente, fácil de estender sem medo de quebrar algo existente.

Parte 3 - Respostas às Questões Analíticas

1. **Abstrações e divisão em camadas** O projeto ficou separado em camadas bem definidas:
 - **Domínio** (com.infnet.domain): só a classe Entrega, imutável, com validação no construtor.
 - **Regras de negócio** (com.infnet.domain): interface CalculadoraFrete + as estratégias FreteExpresso, FretePadrao, FreteEconomico e também a interface PromocaoFrete com PromocaoPesoAlto.
 - **Serviço/aplicação** (com.infnet.service): EtiquetaService apenas orquestra, recebe as estratégias por injeção e gera etiquetas e resumos.

O bloco gigante de if-else desapareceu por completo. Agora o comportamento correto vem por injeção ou pela FreteFactory. Para criar um novo tipo de frete ou promoção basta adicionar uma nova classe que implemente a interface – nada mais precisa ser alterado. Isso cumpre 100% o princípio Aberto-Fechado (OCP).

A divisão traz três grandes vantagens:

- código mais fácil de entender (cada pacote tem um único motivo para existir);
 - mudanças em regras de frete não quebram etiquetas;
 - testes ficam simples, pois cada camada pode ser testada isoladamente com mocks.
2. **Contratos e integridade do domínio** Entrega é imutável e valida tudo logo no construtor. Se algum dado estiver errado, lança na hora uma das exceções específicas:
 - InvalidPesoException
 - InvalidEnderecoException
 - InvalidTipoFreteException

Com isso não existe mais objeto em estado inválido rodando pelo sistema e também não tem mais “frete zero silencioso” quando o tipo é desconhecido. A falha acontece cedo e de forma explícita (fail-fast), o que deixa o sistema muito mais previsível e seguro.

3. **Nomenclatura e estrutura do código** Todos os nomes foram escolhidos para deixar a intenção óbvia:
 - Classe principal passou a se chamar Entrega (mais próximo do que realmente representa).
 - Atributos são privados com getters (encapsulamento total).
 - Interface virou CalculadoraFrete – verbo + substantivo deixa claro o que ela faz.
 - Estratégias têm nomes diretos: FreteExpresso, FretePadrao, FreteEconomico.
 - Constantes ganharam nomes significativos: TAXA_POR_KG, TAXA_FIXA, LIMITE_GRATIS.
 - Promoção chama-se PromocaoPesoAlto – já diz a condição da regra.
 - Métodos do serviço são imperativos: gerarEtiqueta(), gerarResumoPedido().

Cada método é curto (máximo ~15 linhas) e faz apenas uma coisa, seguindo o Single Responsibility Principle na prática.

4. Organização de pacotes

com.infnet

|— domain → entidades e validações

|— exception → exceções do domínio

|— frete → todas as estratégias de frete e promoções

|— service → orquestração e geração de textos

Alta coesão dentro do pacote, baixo acoplamento entre eles. Quem chega novo no projeto acha rapidinho onde cada coisa está e onde deve colocar uma nova regra.

5. **Clareza que elimina comentários** Com nomes bem escolhidos e classes pequenas, o código explica sozinho o que está acontecendo. Exemplo:

if (calculadoraFrete.isFreteGratis(entrega)) { ... }

Ninguém precisa de comentário para entender – o nome da classe e do método já dizem tudo. Quase não há comentários no código (só JavaDoc nas partes públicas). O código ficou autoexplicativo, reduzindo o trabalho de quem lê e o risco de comentário ficar desatualizado.